

Muter user in voicemeet socket:

```
socket.on("muteUserInVoicemeet", async (data) => {  
  try {  
    var { userid, streamid, muteTime } = data;
```

The streamid is the voicemeetid and muteTime is optional boolean if you want to mute the user for 5 minutes then make it true otherwise false. The userId the id of the user you want to mute or unmute. Once called mutes the user called again make the user unmute.

If this muteTime is true, then after 5 minutes the user will be unmuted automatically and all the users will get the same socket as below.

For this socket all the members of the voicemeet will get this response in socket "userMute" the data in variable response in screenshot below:

```
const response = {  
  streamid,  
  action,  
  userid,  
  liveRemoteId: user.liveRemoteId,  
  username: user.username,  
  profilePic,  
};  
  
var members = stream.members;  
for (const member of members) {  
  const sockets = onlineSockets.get(member.toString());  
  if (sockets) {  
    for (const socket of sockets) {  
      if (socket) {  
        socket.emit("userMute", response);  
      }  
    }  
  }  
}
```

Making a user moderator in voicemeet:

```
socket.on("makeModeratorInVoicemeet", async (data) => {  
  try {  
    const { streamid, userid } = data;
```

Here, streamid is the voicemeet and the userId is the id of the user you want to make moderator.

All the members in the voicemeet will receive this following socket "newModerator" with data in the newModerator variable in screenshot below:

```
const newModerator = {  
  _id: user._id,  
  username: user.username,  
  profilePic: await getPicUrl(user._id),  
};  
  
for (const member of members) {  
  const sockets = onlineSockets.get(member.toString());  
  
  if (sockets) {  
    for (const socket of sockets) {  
      if (socket) {  
        socket.emit("newModerator", newModerator);  
      }  
    }  
  }  
}
```

Removing moderator in voicemeet:

```
socket.on("removeModeratorInVoicemeet", async (data) => {  
  try {  
    const { streamid, userid } = data;
```

Data same as

above sockt, I return removedModerator socket:

```
const newModerator = {  
  _id: user._id,  
  username: user.username,  
  profilePic: await getPicUrl(user._id),  
};  
  
for (const member of members) {  
  const sockets = onlineSockets.get(member.toString());  
  
  if (sockets) {  
    for (const socket of sockets) {  
      if (socket) {  
        socket.emit("removedModerator", newModerator);  
      }  
    }  
  }  
}
```

Block user in voicemeet:

```
socket.on("blockInVoicemeet", async (data) => {  
  try {  
    const { streamid, userid } = data;  
    const stream = await VoiceMeet.findById(streamid);
```

It return "newBlockInStream":

```
const newBlocked = {  
  _id: user._id,  
  username: user.username,  
  profilePic: await getPicUrl(user._id),  
};  
  
for (const member of members) {  
  const sockets = onlineSockets.get(member.toString());  
  
  if (sockets) {  
    for (const socket of sockets) {  
      if (socket) {  
        socket.emit("newBlockInStream", newBlocked);  
      }  
    }  
  }  
}
```

Unblock user in voicemeet:

```
socket.on("unblockInVoicemeet", async (data) => {  
  try {  
    const { streamid, userid } = data;
```

It return this socket to all members in voice meet

```
const newUnblocked = {  
  _id: user._id,  
  username: user.username,  
  profilePic: await getPicUrl(user._id),  
};  
  
for (const member of members) {  
  const sockets = onlineSockets.get(member.toString());  
  
  if (sockets) {  
    for (const socket of sockets) {  
      if (socket) {  
        socket.emit("newUnblockInStream", newUnblocked);  
      }  
    }  
  }  
}
```

Send stream action: (hand raise, like, thumbsup)

```
socket.on("actionSendInVoiceMeet", async (data) => {  
  try {  
    var { streamid, action, userid } = data;  
    var stream = await VoiceMeet.findById(streamid);
```

This socket will send socket to all the members in voicemeet his socket
“streamActionReceive” where the action will be same as went in this socket:

```
const response = {  
  streamid,  
  action,  
  userid: moderator._id,  
  username: moderator.username,  
  profilePic,  
  liveRemoteId: moderator.liveRemoteId,  
};  
  
for (const member of members) {  
  const sockets = onlineSockets.get(member.toString());  
  if (sockets) {  
    for (const socket of sockets) {  
      if (socket) {  
        socket.emit("streamActionReceive", response);  
      }  
    }  
  }  
}
```